ROOTCAUSE SERVER

14 · Solución de problemas

Documento: 14-troubleshooting.md **Versión analizada:** 0.2.0 **Commit:** 2efedb2 **Fecha del análisis:** 2026-08-26

CONTENIDO

Índice por área
Instalación y compilación
- La compilación falla con un error del enlazador
- cargo usa una versión de Rust distinta a la esperada
- cargo build recompila todo cada vez en Docker

Arranque del servidor
- falta ROOTCAUSE_API_TOKEN; genera uno con 'rootcause-server token'
- ROOTCAUSE_API_TOKEN no debe empezar ni terminar con espacios
- el modo de desarrollo inseguro solo puede escuchar en una dirección de loopback
- --trust-forwarded-for solo tiene sentido detrás de un proxy inverso
- el almacenamiento 0.2 requiere una URL sqlite:
- política inválida: …
- no se pudo enlazar RootCause Server a 127.0.0.1:8080

Conexión y autenticación
- Todo devuelve 401 unauthorized
- 429 con código locked_out
- 429 con código rate_limited
- Detrás del proxy, todas las peticiones parecen venir de la misma dirección

El agente
- no se enviará un token por HTTP remoto
- registra el activo antes de enviar telemetría (404)
- La telemetría se acepta pero la superficie llega vacía
- demasiados sockets en escucha en un solo envío (400)
- 413 Payload Too Large
- El agente reintenta y no se recupera
- Los archivos vigilados por omisión dejaron de vigilarse

Detección e incidentes
- Aparece un hallazgo de reloj desincronizado nada más enrolar
- Un servidor web de borde genera hallazgos por los puertos 80 y 443
- Un puerto se cerró pero el incidente sigue abierto
- Un incidente marcado como reconocido vuelve a abrirse
- Falsos positivos de silencio de agente
- La regla de salida anómala no se dispara nunca
- La CPU está al 100 % y no aparece nada

Consola
- La consola pide el token una y otra vez
- La consola no carga y el navegador reporta errores de CSP
- Aviso amarillo permanente en la parte superior
- Una ruta desconocida devuelve la consola en vez de 404

Base de datos
- database migration failed
- database is locked
- El archivo crece más de lo esperado
- Faltan datos históricos

Despliegue y contenedor
- El contenedor arranca y muere

El healthcheck falla aunque el servidor responde
El servicio de systemd no arranca
Un agente en macOS no reporta actualizaciones pendientes
CI y pruebas
guard_claims.py falla
guard_encoding.py falla
El trabajo «Todo en verde» falla y todos los demás están bien
cargo audit falla por RUSTSEC-2023-0071
scripts/smoke.sh falla en local pero CI está verde
Comandos de diagnóstico rápido

> Guía práctica organizada por síntoma. Cada entrada indica la causa probable, cómo diagnosticarla, cómo resolverla, qué archivos intervienen y **qué riesgo tiene la solución**.
>
> Los mensajes citados son literales del código. Cualquiera se puede localizar con `grep` en el repositorio.

Índice por área

- Instalación y compilación
- Arranque del servidor
- Conexión y autenticación
- El agente
- Detección e incidentes
- Consola
- Base de datos
- Despliegue y contenedor
- CI y pruebas

Instalación y compilación

La compilación falla con un error del enlazador

Síntoma	`error: linker 'link.exe' not found`, o similar
Causa	Falta el enlazador nativo de la plataforma
Diagnóstico	`rustc --version --verbose` y comprobar el toolchain de C++ del sistema
Solución	Windows: instalar las *Build Tools* de Visual Studio con la carga de trabajo de C++. Linux: `build-essential`. macOS: `xcode-select --install`. Alternativa en Windows: compilar dentro de WSL
Archivos	`rust-toolchain.toml`
Riesgo	Ninguno

`cargo` usa una versión de Rust distinta a la esperada

Síntoma	Errores de sintaxis de la edición 2024, o características no reconocidas

Causa	`rustup` no aplicó el canal fijado
Diagnóstico	`rustc --version` — debe decir `1.95.0`
Solución	`rustup toolchain install 1.95.0`, ejecutando `cargo` desde la raíz del repositorio para que `rust-toolchain.toml` tenga efecto
Archivos	`rust-toolchain.toml`, `Cargo.toml` (`rust-version`)
Riesgo	Ninguno

`cargo build` recompila todo cada vez en Docker

Síntoma	Cada `docker build` tarda lo mismo aunque solo cambie una línea
Causa	El truco de caché del `Dockerfile` se rompió: `COPY crates` antes de `cargo fetch`
Diagnóstico	Revisar el orden de `COPY` en el `Dockerfile`
Solución	Mantener el orden: manifiestos → `cargo fetch --locked` → `crates/` → `console/` → `cargo build`
Archivos	`Dockerfile`
Riesgo	Ninguno

Arranque del servidor

`falta ROOTCAUSE_API_TOKEN; genera uno con 'rootcause-server token'`

Causa	Se ejecutó `serve` sin token y sin `--insecure-dev-mode`
Diagnóstico	`echo $ROOTCAUSE_API_TOKEN` (Bash) o `$env:ROOTCAUSE_API_TOKEN` (PowerShell)
Solución	`rootcause-server token`, exportar el valor y volver a arrancar. En desarrollo, `bash scripts/dev.sh` lo hace por ti
Archivos	`config.rs`
Riesgo	Ninguno. **No** resolverlo con `--insecure-dev-mode` fuera de desarrollo

`ROOTCAUSE_API_TOKEN no debe empezar ni terminar con espacios`

Causa	Un espacio o salto de línea al copiar y pegar	
Diagnóstico	`printf '%s' "$ROOTCAUSE_API_TOKEN" \	wc -c` — deben ser 64 caracteres exactos si viene de token`
Solución	Recortar el valor	
Riesgo	Ninguno. El servidor rechaza esto **a propósito**, para evitar un desajuste inexplicable después	

`el modo de desarrollo inseguro solo puede escuchar en una dirección de loopback`

Causa	`--insecure-dev-mode` combinado con un enlace público
Solución	Usar token de verdad, o volver a `127.0.0.1`
Riesgo	**No forzarlo.** Un plano de control sin autenticación en una red es un regalo

`--trust-forwarded-for solo tiene sentido detrás de un proxy inverso`

Causa	La opción activada con enlace en loopback
Solución	Quitar la opción, o enlazar fuera de loopback **después** de poner el proxy
Riesgo	Activarla sin proxy permite que cualquiera elija qué dirección se limita y se bloquea

`el almacenamiento 0.2 requiere una URL sqlite:`

Causa	Se pasó una URL de PostgreSQL o MySQL
Solución	Usar `sqlite://ruta.db`. Esta versión no admite otro motor
Riesgo	Ninguno

`política inválida: …`

Causa	El archivo de `--policy-file` es contradictorio o tiene un campo desconocido
Diagnóstico	`rootcause-server policy --file politica.json` — imprime el error exacto
Solución	Corregir según el mensaje. Partir de `rootcause-server policy > politica.json`
Archivos	`policy.rs`
Riesgo	Ninguno. Que el arranque falle es deliberado: una política que no puede dispararse parece defensa y no la es

`no se pudo enlazar RootCause Server a 127.0.0.1:8080`

Causa	Puerto ocupado, o sin permiso (puertos < 1024 en Unix)		
Diagnóstico	`ss -tlnp \`	grep 8080 `en Linux;` netstat -ano \	findstr 8080` en Windows
Solución	Cambiar `--bind`, o liberar el puerto		
Riesgo	Ninguno		

Conexión y autenticación

Todo devuelve `401 unauthorized`

Diagnóstico	`curl -s -H "Authorization: Bearer $ROOTCAUSE_API_TOKEN" http://127.0.0.1:8080/api/v1/status`
Causas	Token distinto entre cliente y servidor · esquema mal escrito (debe ser la palabra `Bearer`, un espacio y el token) · espacio envolvente
Solución	Reexportar el mismo token en ambos lados
Archivos	`auth.rs`
Riesgo	**Cuidado:** cada intento fallido acerca al bloqueo. Diez fallos y esa dirección queda fuera 300 s

`429` con código `locked_out`

Causa	Se alcanzó el umbral de fallos desde esa dirección

Diagnóstico	La cabecera `Retry-After` dice cuántos segundos faltan; `GET /api/v1/threats` muestra el contador `auth.lockout`
Solución	Esperar. Con urgencia justificada, **reiniciar el servidor** levanta todos los bloqueos, porque el estado del perímetro es solo memoria
Riesgo	Reiniciar también levanta los bloqueos de quien esté atacando. Hacerlo solo si es un bloqueo propio

`429` con código `rate_limited`

Causa	La cubeta de esa dirección se vació
Diagnóstico	Contar peticiones por minuto desde esa dirección. **Si todos los agentes salen tras el mismo NAT, comparten cubeta**
Solución	Subir `--rate-limit-per-minute`, o espaciar los ciclos del agente
Riesgo	Un límite muy alto debilita la protección contra fuerza bruta

Detrás del proxy, todas las peticiones parecen venir de la misma dirección

Causa	`--trust-forwarded-for` no está activado, así que se usa la dirección del proxy
Diagnóstico	Revisar el contenido de `defense_events.source`
Solución	Activar `--trust-forwarded-for` **y** asegurarse de que el proxy fija `X-Forwarded-For`
Riesgo	**Alto si no hay proxy real.** Cualquiera podría suplantar la dirección y provocar el bloqueo de un tercero

---

El agente

`no se enviará un token por HTTP remoto`

Causa	`--server-url` apunta a un `http://` que no es loopback
Solución	Usar `https://` con el proxy delante. Solo en laboratorio, `--allow-insecure-http`
Archivos	`main.rs`, `validate_transport`
Riesgo	Forzarlo pone el token en claro en la red

`registra el activo antes de enviar telemetría` (404)

Causa	Llegó telemetría de un `agent_id` que no está en `assets`
Diagnóstico	`GET /api/v1/assets` y buscar el identificador
Solución	El agente registra antes de enviar en cada ciclo; si aparece, revisar que el registro no esté fallando por validación (nombre de equipo, versión, arquitectura o etiquetas fuera de límites)
Riesgo	Ninguno

La telemetría se acepta pero la superficie llega vacía

Síntoma	La consola muestra el equipo sin puertos ni eventos de autenticación
Causa	Las sondas fallaron: falta `ss`, `journalctl`, `ufw`… o faltan privilegios
Diagnóstico	**Leer las advertencias.** El agente las registra con `warn!` y el servidor las devuelve en `warnings`. También aparecen en la consola como superficies no inspeccionadas

Solución	Instalar la herramienta que falta, o dar al agente permiso de lectura. En Linux, `journalctl` suele necesitar pertenecer al grupo `systemd-journal`
Archivos	`probe.rs`, `net.rs`
Riesgo	Ninguno. **Esto es el sistema funcionando bien**: declara la brecha en lugar de reportar un cero

`demasiados sockets en escucha en un solo envío` (400)

Causa	Más de 1 024 sockets en un sobre	
Diagnóstico	`ss -tuln \	wc -l` en el equipo
Solución	Investigar por qué el equipo tiene tantos sockets: podría ser el incidente. El agente ya recorta a 256 y declara la brecha, así que llegar a 1 024 apunta a un agente manipulado	
Riesgo	Subir el límite ocultaría la anomalía	

`413 Payload Too Large`

Causa	El sobre supera `--max-body-kib` (1 MiB por omisión)
Solución	Subir el límite hasta 16 384 KiB, o reducir la lista de archivos vigilados
Riesgo	Un límite muy alto permite que un agente comprometido consuma memoria

El agente reintenta y no se recupera

Síntoma	`no se pudo entregar la telemetría` cada vez con más espera
Causa	Reintento exponencial hasta 300 s
Diagnóstico	`curl http://servidor:8080/healthz` desde el mismo equipo
Solución	Arreglar la conectividad. El agente vuelve al intervalo normal al primer éxito
Riesgo	**Los ciclos perdidos no se reenvían**: no hay cola local

Los archivos vigilados por omisión dejaron de vigilarse

Causa	Se usó `--watch-list`, que **reemplaza** los valores por omisión de la plataforma
Diagnóstico	`rootcause-agent --dry-run` y revisar `security.watched_files`
Solución	Usar `--watch-file` (repetible), que **complementa** en lugar de reemplazar
Archivos	`integrity.rs`
Riesgo	Sin corregirlo, `sshd_config` y `sudoers` quedan sin vigilancia

Detección e incidentes

Aparece un hallazgo de reloj desincronizado nada más enrolar

Causa	Más de 120 s entre `observed_at` del agente y `received_at` del servidor
Diagnóstico	Comparar `date -u` en ambos equipos

Solución	Sincronizar por NTP. Si la latencia real lo justifica, subir `clock_skew_seconds` en la política
Riesgo	Una tolerancia amplia dificulta correlacionar registros entre equipos

Un servidor web de borde genera hallazgos por los puertos 80 y 443

Causa	El rol no se declaró; por omisión es `internal-server`
Solución	Reenrolar con `--label role=edge`. También se puede añadir el puerto a `public_allowlist` en la política
Archivos	`policy.rs`, `is_allowed_public`
Riesgo	Ampliar la lista de permitidos silencia esos puertos **en todos los roles**

Un puerto se cerró pero el incidente sigue abierto

Causa probable	En ese ciclo hubo brecha en `listeners`, así que la categoría `Exposure` **no** se auto-resolvió
Diagnóstico	Revisar `uninspected_surfaces` en la postura del activo, o las `warnings` de la ingesta
Solución	Arreglar la sonda para que el ciclo siguiente inspeccione de verdad. También se puede marcar el incidente como resuelto a mano, y queda auditado
Archivos	`api.rs`, `self_healing_categories`
Riesgo	Ninguno. Es la protección de REQ-SEC-003 haciendo su trabajo

Un incidente marcado como reconocido vuelve a abrirse

Causa	La condición se sigue observando. `upsert_incident` reabre a propósito: reconocer no es arreglar
Solución	Corregir la causa. Si es un falso positivo, ajustar la política o abrir una incidencia con la plantilla `falso_positivo.yml`
Riesgo	Ninguno

Falsos positivos de silencio de agente

Causa	`--agent-interval-seconds` del **servidor** no coincide con el del agente
Diagnóstico	Comparar ambos valores
Solución	Igualarlos. Alternativa: subir `silence_intervals` en la política
Riesgo	Un valor alto retrasa la detección de un agente realmente silenciado

La regla de salida anómala no se dispara nunca

Causa	Necesita **al menos 3 muestras** de historial y una tasa por encima de 1 MB/s
Diagnóstico	`GET /api/v1/assets/{id}` y contar el historial
Solución	Esperar a acumular muestras. Ajustar `egress_floor_bytes_per_second` si el suelo no encaja con el enlace real
Riesgo	Bajar el suelo dispara la regla con tráfico normal

La CPU está al 100 % y no aparece nada

Causa	Hace falta una **racha** de `cpu_sustained_samples` (3) muestras consecutivas
Solución	Esperar. Un pico aislado no es saturación, y reportarlo enseñaría a ignorar la alerta

Riesgo	Bajar a 1 llena la consola de ruido

Consola

La consola pide el token una y otra vez

Causa	Se guarda en `sessionStorage`: se pierde al cerrar la pestaña
Solución	Ninguna; es intencional
Riesgo	Cambiarlo a `localStorage` dejaría el token en el disco del navegador

La consola no carga y el navegador reporta errores de CSP

Causa	Alguien añadió script en línea o un recurso externo
Diagnóstico	`python3 scripts/guard_console.py` — señala el archivo y la línea
Solución	Construir el nodo con la API del DOM en lugar de con una cadena; servir el recurso desde el propio origen
Archivos	`headers.rs`, `app.js`
Riesgo	Relajar la CSP eliminaría la primera capa de defensa contra XSS

Aviso amarillo permanente en la parte superior

Causa	`hardening` declara algo que debilita la postura: sin autenticación, o enlace fuera de loopback
Diagnóstico	`GET /api/v1/status`, bloque `hardening`
Solución	Poner token, o volver a loopback tras un proxy
Riesgo	Ninguno. El aviso es honestidad, no un fallo

Una ruta desconocida devuelve la consola en vez de `404`

Causa	Comportamiento intencional para que funcione la navegación del lado del cliente
Nota	Las rutas reservadas (`api/`, `healthz`, `readyz`, `metrics`) **sí** devuelven `404`
Archivos	`ui.rs`

Base de datos

`database migration failed`

Causa	Base modificada a mano, o de una versión más nueva que el binario
Diagnóstico	`sqlite3 rootcause.db "SELECT * FROM _sqlx_migrations;"`
Solución	Restaurar un respaldo coherente con la versión del binario
Riesgo	**Editar `_sqlx_migrations` a mano deja la base en un estado que nadie puede razonar.** No hacerlo

`database is locked`

Causa	Otro proceso escribe, o la espera de 10 s se agotó
Diagnóstico	Comprobar que no hay dos instancias apuntando al mismo archivo
Solución	Ejecutar una sola instancia. **SQLite no admite dos escritores concurrentes**, y no hay alta disponibilidad en esta versión
Riesgo	Dos instancias sobre el mismo archivo pueden corromper datos

El archivo crece más de lo esperado

Causa	Retención larga, muchos activos, o intervalo corto
Diagnóstico	`sqlite3 rootcause.db "SELECT COUNT(*) FROM telemetry;"`
Solución	Bajar `--retention-days`, o subir el intervalo del agente. Tras podar, `VACUUM` recupera el espacio del archivo
Riesgo	`VACUUM` **bloquea la base mientras se ejecuta.** Hacerlo en ventana de mantenimiento
Nota	Los incidentes y la auditoría **no se podan nunca**. Si el crecimiento viene de ahí, es evidencia y hay que archivarla, no borrarla

Faltan datos históricos

Causa	La retención los eliminó
Solución	Nada retroactivo. Para el futuro, subir `--retention-days` y exportar periódicamente con `GET /api/v1/export`
Riesgo	Ninguno

Despliegue y contenedor

El contenedor arranca y muere

Diagnóstico	`docker compose logs rootcause-server`
Causa frecuente 1	Falta `ROOTCAUSE_API_TOKEN`. Compose lo dice: *«Genera uno con: rootcause-server token»*
Causa frecuente 2	El volumen no permite escribir en `/var/lib/rootcause`, con `read_only: true` en el resto
Solución	Exportar el token; comprobar que el volumen está montado y pertenece al UID 10001
Riesgo	**No quitar** `read_only` para "arreglarlo": rompe el endurecimiento

El *healthcheck* falla aunque el servidor responde

Causa	El chequeo usa `/dev/tcp/127.0.0.1/8080`; si se cambió `ROOTCAUSE_BIND` dentro del contenedor, ya no coincide
Solución	Mantener `ROOTCAUSE_BIND=0.0.0.0:8080` dentro del contenedor y publicar el puerto solo en `127.0.0.1` del anfitrión
Archivos	`compose.yml`
Riesgo	Publicar en `0.0.0.0` en el anfitrión expone el plano de control

El servicio de systemd no arranca

Diagnóstico	`systemctl status rootcause-server` y `journalctl -u rootcause-server -n 50`
Causas	Falta `/etc/rootcause/server.env` · el usuario `rootcause` no existe · `/var/lib/rootcause` no le pertenece

Solución	Crear usuario y directorio; el archivo de entorno debe ser de `root` con modo `0600`
Riesgo	**No relajar `ProtectSystem=strict` ni `CapabilityBoundingSet=`** para depurar

Un agente en macOS no reporta actualizaciones pendientes

Causa	El conteo de actualizaciones **solo está implementado en Linux**
Diagnóstico	La brecha lo dice: *«el conteo de actualizaciones de seguridad solo está implementado en Linux»*
Solución	Ninguna. Es un límite declarado en `docs/CROSS_PLATFORM.md`
Riesgo	Ninguno; aparece como superficie no inspeccionada, no como cero

---

CI y pruebas

`guard_claims.py` falla

Causa	Se añadió o quitó una regla y algún documento sigue diciendo el número anterior; o una acción de GitHub no está pinneada a SHA de 40 caracteres
Diagnóstico	El script nombra el archivo y el valor esperado
Solución	Actualizar los documentos, o pinnear la acción
Riesgo	Ninguno. Falla **para eso**

`guard_encoding.py` falla

Causa	Un archivo con BOM, con CRLF o con mojibake
Solución	Reguardar en UTF-8 sin BOM con saltos LF. `.gitattributes` y `.editorconfig` lo previenen
Riesgo	Ninguno

El trabajo «Todo en verde» falla y todos los demás están bien

Causa	Algún trabajo quedó `skipped` o `cancelled`. La puerta exige `success` en los catorce
Solución	Relanzar el trabajo que no se ejecutó
Riesgo	Ninguno. **Un control que no se ejecutó no cuenta como aprobado**

`cargo audit` falla por RUSTSEC-2023-0071

Causa	La exención dejó de aplicarse, o `cargo tree --invert rsa` empezó a devolver algo
Diagnóstico	`cargo tree --invert rsa`. Si devuelve una cadena de dependencias, **`rsa` sí se está compilando**
Solución	Investigar qué feature activó el driver MySQL de `sqlx` y desactivarla. **La exención no se amplía: se corrige la causa**
Archivos	`.cargo/audit.toml`, `deny.toml`
Riesgo	Ampliar la exención sin verificar la convertiría en una excusa

`scripts/smoke.sh` falla en local pero CI está verde

Causas	El puerto 18080 está ocupado · falta `curl` · no hay Bash

Solución	`ROOTCAUSE_SMOKE_PORT=18099 bash scripts/smoke.sh`
Riesgo	Ninguno

Comandos de diagnóstico rápido

Estado general del plano de control:

```
curl -s -H "Authorization: Bearer $ROOTCAUSE_API_TOKEN" http://127.0.0.1:8080/api/v1/status
```

Qué enviaría un agente, sin enviarlo:

```
rootcause-agent --dry-run
```

Catálogo de reglas que implementa el binario:

```
rootcause-server rules
```

Validar una política antes de desplegarla:

```
rootcause-server policy --file politica.json
```

Incidentes críticos abiertos:

```
curl -s -H "Authorization: Bearer $ROOTCAUSE_API_TOKEN" "http://127.0.0.1:8080/api/v1/incidents?status=open&severity=critical"
```

Qué está rechazando el propio perímetro:

```
curl -s -H "Authorization: Bearer $ROOTCAUSE_API_TOKEN" http://127.0.0.1:8080/api/v1/threats
```

Paquete de evidencia completo:

```
curl -s -H "Authorization: Bearer $ROOTCAUSE_API_TOKEN" -o evidencia.ndjson http://127.0.0.1:8080/api/v1/export
```

Tamaño de las tablas que crecen:

```
sqlite3 rootcause.db "SELECT 'telemetry', COUNT(*) FROM telemetry UNION ALL SELECT 'incidents', COUNT(*) FROM incidents UNION ALL SELECT 'auth_pressure', COUNT(*) FROM auth_pressure;"
```

Anterior: 13 · Despliegue y operación · **Siguiente:** 15 · Riesgos y deuda técnica · **Índice:** README